

Graph Coloring via Metaheuristic Optimization

GA · GAE · SA · ACO · Hybrid GA+SA · Hybrid ACO+SA

Elena Aschmann Mina Camenisch

Optimization Methods for Engineers, 227-0707-00L
ETH Zürich

Spring 2026

Problem: Graph Coloring

Given an undirected graph $G = (V, E)$

Find an assignment $c : V \rightarrow \{1, \dots, k\}$ such that

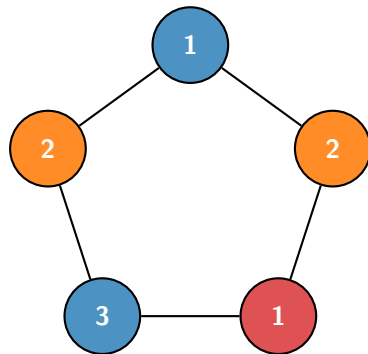
$$\forall (u, v) \in E : c(u) \neq c(v)$$

Minimize k — the *chromatic number* $\chi(G)$

Why it's hard

$\chi(G)$ is NP-hard to compute exactly. Brute force:
 $O(k^{|V|})$.

Applications: register allocation, scheduling,
frequency assignment, map colouring



C_5 : 3-coloring, $\chi(C_5) = 3$

Mathematical Formulation

Search Space

$$\mathcal{S} = \{1, \dots, k_{\max}\}^{|V|}, \quad |\mathcal{S}| = k_{\max}^{|V|}$$

Fitness Function

$$F(c) = \underbrace{\alpha \cdot v_{\text{viol}}(c)}_{\text{hard: feasibility}} + \underbrace{\beta \cdot k_{\text{used}}(c)}_{\text{soft: minimise colours}}$$

$$\alpha = 1000, \quad \beta = 1$$

$$\text{where } v_{\text{viol}}(c) = |\{(u, v) \in E : c(u) = c(v)\}|$$

$\alpha \gg \beta$ ensures any feasible solution beats any infeasible one.

Chromatic Gap (primary metric)

$$\Delta k = k_{\text{used}}(c^*) - \chi(G) \geq 0$$

Lower is better. $\Delta k = 0$ is optimal.

In practice: $\chi(G)$ unknown for random graphs $\Rightarrow$ use DSATUR as reference.

Baselines

BF	Exact; exhaustive; $O(k_{\max}^n)$
DSATUR	Greedy; fast; reference for Δk

Algorithms at a Glance

Algorithm	Search strategy	State space	Init	Key idea
GA	Population	$\{1, \dots, k\}^n$	Random	Crossover + mutation
GAE	Population	$\{1, \dots, k\}^n$	Random	GA + elitism
SA	Single solution	$\{1, \dots, k\}^n$	Random	Metropolis acceptance + restarts
ACO	Colony	$\{1, \dots, k\}^n$	Pheromone	Constructive; pheromone learning
GA+SA	Population + local	$\{1, \dots, k\}^n$	Random	GA search, SA refines elites
ACO+SA	Colony + local	$\{1, \dots, k\}^n$	Pheromone	ACO construction, SA refines best
DSATUR	Greedy	—	—	Baseline; max saturation-degree first
BF	Exhaustive	—	—	Optimal; $n \leq 10$ only

All metaheuristics share the same fitness F and the same interface:

$\text{run}(G, k_{\max}, \text{params}, \text{seed}) \rightarrow \text{AlgoResult}$

Encoding: chromosome $c \in \{1, \dots, k_{\max}\}^n$, gene i
= colour of vertex i

- 1 **Initialise** N_{pop} random individuals
- 2 **Evaluate** $F(c)$ for each
- 3 **Repeat** for N_{gen} generations:
 - Tournament selection (size t)
 - Two-point crossover with prob p_{cx}
 - Per-gene mutation (uniform-int) with prob $p_{\text{mut}} \cdot p_{\text{ind}}$
 - **[GAE] Reinject top n_{elite} individuals**
- 4 **Return** best individual

Tuned parameters

Parameter	GA	GAE
N_{pop}	100	100
p_{cx}	0.90	1.00
p_{mut}	0.50	0.50
p_{ind}	0.05	0.05
N_{gen}	200	200
n_{elite}	—	3
t	5	5

Tuned via OFAT sweep on $n \in \{40, 50, 60\}$, $p = 0.5$

SA: Simulated Annealing with Restarts

① **Initialise** randomly; set $T \leftarrow T_0$

② **Repeat** for $N_{\max}$ moves:

- Propose: reassign one vertex i to colour $c' \neq c(i)$
- Compute ΔF incrementally in $O(\deg(i))$
- Accept with probability

$$P = \min\left(1, e^{-\Delta F/T}\right)$$

- Cool: every n_{step} moves, $T \leftarrow \gamma T$
- **Restart** to best-so-far if no improvement for n_{stall} accepted moves

③ **Return** global best

Tuned parameters

Parameter	Value
T_0	100
γ	0.999
n_{step}	$5n$
n_{stall}	100
$N_{\max}$	$200 \cdot n_{\text{step}}$

Key property: incremental ΔF update makes each move $O(\deg(i))$ rather than $O(|E|)$.
Enables large $N_{\max}$ within tight time budget.

ACO: Ant Colony Optimisation

Pheromone matrix $\tau \in \mathbb{R}^{|V| \times k_{\max}}$: τ_{ic} = desirability of assigning colour c to vertex i

Heuristic (conflict avoidance):

$$\eta_{ic} = \frac{1}{1 + |\{j \in N(i) : c(j) = c\}|}$$

Each ant constructs a full colouring:

$$P(i, c) \propto \tau_{ic}^{\alpha} \cdot \eta_{ic}^{\beta}$$

visiting vertices in a *randomly shuffled* order.

Pheromone update (best-ant only):

$$\tau \leftarrow (1 - \rho) \tau + \frac{Q}{F_{\text{best}}} \cdot \mathbf{1}[\text{best ant}]$$

Tuned parameters

Parameter	Value
N_{ants}	200
α	2.0
β	5.0
ρ	0.1
Q	1.0
$\tau_{\min}$	0.01
N_{iter}	300

High β prioritises the heuristic (greedy conflict avoidance); each ant uses an **independent** vertex permutation for solution diversity.

Hybrid GA+SA and Hybrid ACO+SA

GA+SA: standard GA population search; each generation, the elite individual(s) undergo a short SA local-search refinement pass before re-entering the population.

ACO+SA: standard ACO pheromone-guided construction; once ants finish, the best-so-far colouring is refined by an SA local search with periodic restarts.

Both combine a global population/colony search with a cheap local-move refinement stage, aiming to reach better solutions than GA/ACO alone by polishing promising regions instead of only exploring globally.

Tuned parameters: GA+SA

Parameter	Value
$N_{\text{pop}}, N_{\text{gen}}$	80, 120
$p_{\text{cx}}, p_{\text{mut}}, p_{\text{ind}}$	0.6, 0.4, 0.1
n_{elite}, t	1, 4
SA refine fraction	0.2
SA T_0, γ	1, 0.9

Tuned parameters: ACO+SA

Parameter	Value
$N_{\text{ants}}, N_{\text{iter}}$	20, 100
α, β, ρ	1, 3, 0.2
SA restarts	5
SA T_0, γ	10, 0.9

Experimental Setup

Benchmark graphs

- Erdős–Rényi random graphs $G(n, p)$
- $n \in \{20, 30, 40, 50, 75, 100, 150, 200\}$
- $p \in \{0.3, 0.5, 0.7\}$
- $R = 20$ independent seeds per (n, p)
- Small- n baseline: BF exact for $n \in \{8, 10\}$

Infrastructure

- ETH pira cluster (SLURM)
- 1 job per (n, p) combination (24 parallel jobs)
- 1 job per tuning combination (60 parallel jobs)
- Merging + analysis run locally

Primary metric

$$\text{gap}_{\text{DSATUR}} = k_{\text{used}} - k_{\text{DSATUR}}$$

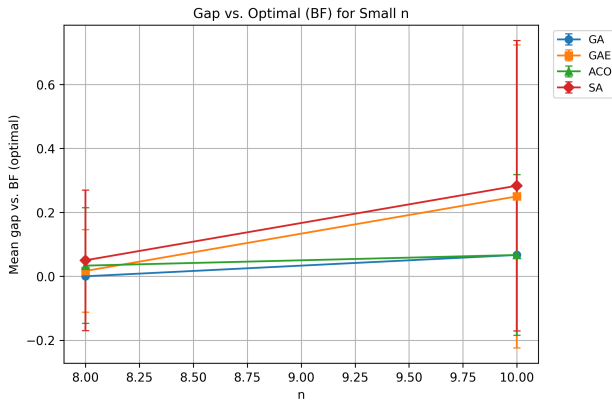
True $\chi(G)$ unknown $\Rightarrow$ DSATUR used as reference.

Negative gap = *better* than DSATUR.

Tuning procedure

- OFAT sweep: one parameter varied at a time
- Fixed base: $n \in \{40, 50, 60\}$, $p = 0.5$, $R = 10$
- Best value selected by lowest mean Δk

Results: Optimality at Small n



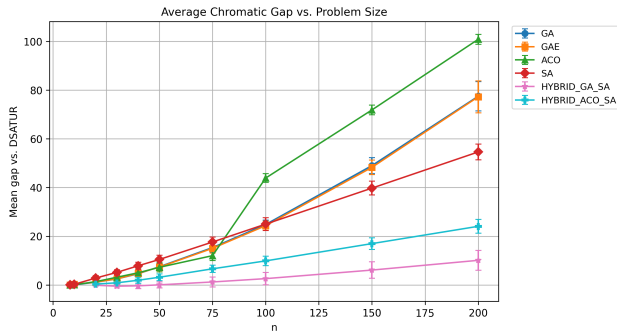
Gap vs. exact optimum (BF) for $n \in \{8, 10\}$

- GA/GAE/SA/ACO all near-optimal; mean gap ≈ 0.1 colours
- GA closest to BF (0.03); SA most variable (0.17, highest std)
- Negative gaps possible: occasionally beats DSATUR-seeded BF reference
- Hybrids not run at this scale (no BF comparison available)

Takeaway

GA, GAE, SA and ACO are all competitive with the exact solver when n is small.

Results: Scalability — Chromatic Gap vs. n



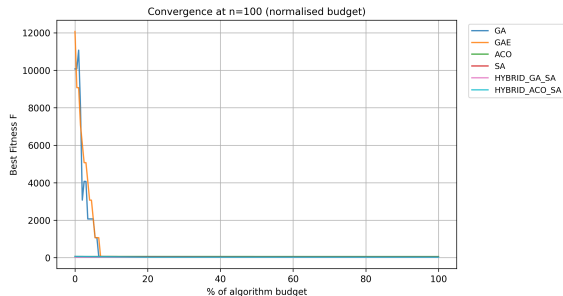
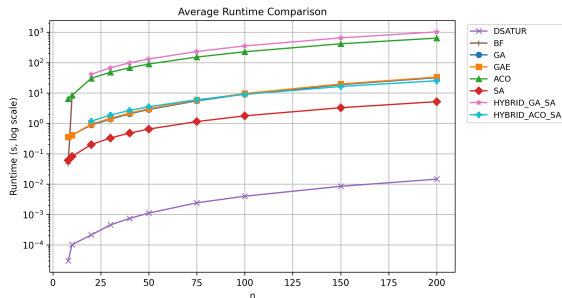
Mean gap $\pm$ std over $p \in \{0.3, 0.5, 0.7\}$, $R = 20$

- **Hybrid GA+SA** best at every n , wide margin
- **Hybrid ACO+SA** distant 2nd, still far ahead of plain ACO
- Originals: **SA** < GA $\approx$ GAE < **ACO**
- Gap grows roughly *linearly* in n for all six; hybrids' slopes are much shallower

At $n = 200$

Algo	Mean gap
Hybrid GA+SA	10.1
Hybrid ACO+SA	24.1
SA	54.6
GAE	77.3
GA	77.5
ACO	100.9

Results: Runtime and Convergence



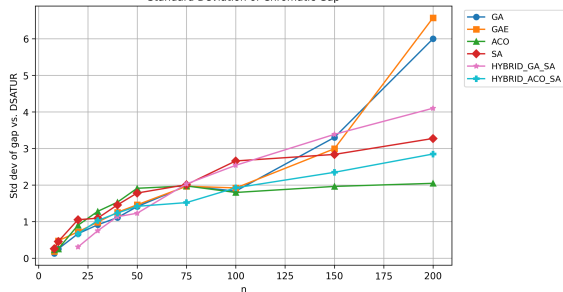
Average wall-clock time, log scale

Best fitness F vs. % of budget ($G(100, 0.5)$)

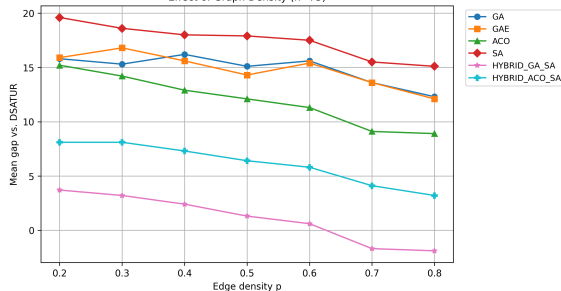
- **DSATUR** 10^3 – $10^5 \times$ faster than any metaheuristic; **SA** fastest metaheuristic (~ 5 s at $n=200$)
- **ACO**, **SA** and **both hybrids** start near-feasible; **GA/GAE** start at $F \approx 11$ – 12 k and need $\sim 8\%$ of budget to reach feasibility, then all plateau early
- **Hybrid GA+SA**: best quality overall but *slowest* algorithm (~ 1026 s at $n=200$) — its per-generation SA pass dominates runtime
- **Hybrid ACO+SA**: better trade-off — $\sim 4 \times$ ACO's quality at $\sim 25\%$ of ACO's runtime (~ 25 s vs. ~ 640 s)

Results: Variance and Effect of Graph Density

Standard Deviation of Chromatic Gap



Effect of Graph Density (n=75)

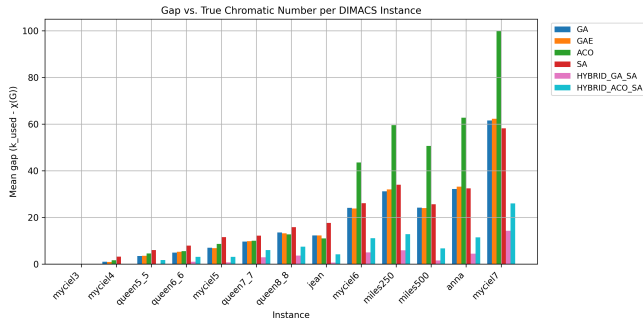


Std. dev. of gap vs. n

- **GAE** now has the highest variance at large n (std ≈ 6.6 at $n=200$), narrowly ahead of **GA** (≈ 6.0) — both unreliable at large n
- **ACO** most consistent (lowest σ) but far from optimal; **Hybrid ACO+SA** nearly as tight while much closer to optimal
- All algorithms improve vs. DSATUR as density p increases; **Hybrid GA+SA** even turns negative (beats DSATUR) for $p \geq 0.7$

Mean gap vs. edge density p ($n=75$)

Results: DIMACS Benchmark (Known $\chi(G)$)



13 fixed instances (myciel3–7, queen5_5–8_8, miles250/500, anna, jean); $\Delta k_{\text{true}} = k_{\text{used}} - \chi(G)$

- **Hybrid GA+SA** best on 12/13 instances; mean $\Delta k_{\text{true}} = 3.1$ vs. 17.3–19.3 for GA/GAE/SA, 28.5 for plain ACO
- **Hybrid ACO+SA** mean $\Delta k_{\text{true}} = 7.2$ — a 4× improvement over plain ACO
- Gap widens most on the largest instance myciel7 ($n=191$): ACO reaches $\Delta k_{\text{true}}=100$ vs. 14.3 for Hybrid GA+SA
- 100% feasibility across all DIMACS runs

Same ranking as the random-graph sweep: hybridising with SA refinement is the single biggest lever on solution quality.

Summary

	GA	GAE	SA	ACO	GA+SA	ACO+SA
Mean gap (all n)	22.8	22.6	20.4	30.7	2.4	8.0
Gap at $n = 200$	77.5	77.3	54.6	100.9	10.1	24.1
Mean runtime (s)	9.1	9.5	1.6	208.9	324.9	8.2
Feasibility	100%	99.8%	100%	100%	100%	100%
Variance (large n)	High	Highest	Med.	Low	Med.	Low

Best quality

Hybrid GA+SA — best mean gap and best at $n = 200$, on both random graphs and DIMACS

Best quality/speed

SA fastest overall; **Hybrid ACO+SA** best trade-off among the refined variants

Conclusions and Outlook

Findings

- SA-based local refinement is the single biggest lever on quality: Hybrid GA+SA cuts the mean gap from 18–23 (GA/GAE/SA) to **2.4**, and the $n=200$ gap from ~ 77 to **10.1**
- Elitism alone (GAE vs. GA) barely moves the needle by comparison
- Wrapping ACO's slow construction in SA refinement (Hybrid ACO+SA) roughly quadruples its quality at $\sim 4\%$ of its runtime
- Same ranking holds on 13 DIMACS instances with known $\chi(G)$: Hybrid GA+SA best on 12/13
- Feasibility 100% for 5/6 algorithms; GAE had 1 infeasible run out of 600 (99.8%)

Directions for improvement

- **GA/GAE**: seed population from DSATUR to eliminate the infeasibility phase
- **ACO**: vectorise construction step (NumPy); current Python loop is the bottleneck
- **Hybrid GA+SA**: cheapen the per-generation SA pass — now the slowest of the six despite the best quality
- **Hybrid ACO+SA**: already a strong cost/quality trade-off; tune `sa_restarts` further for large n
- **All**: larger budget (n_{gen} , N_{iter}) for large n

Code, results and cluster scripts: <https://github.com/easchmann/optimisation-project>